SYNOPSIS OF EXPLORATORY PROJECT (EP)

ON

LEDGERLINE – FINANCE OPERATIONS

AI-powered reconciliation, exception management and investigation support

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

KRITIKA AGGARWAL

Roll Number: 2310991878

Submitted to: Dr. Monika Aggarwal

Designation: Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY

CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

CHITKARA UNIVERSITY

1. ABSTRACT

LedgerLine is an AI-powered finance operations platform designed to organize transaction reconciliation, exception management, risk-based attention prioritization, and evidence-grounded investigation in one workflow. The project addresses a common operational problem: financial teams often work with records originating from different systems, where transaction references, dates, amounts, payment statuses, and ledger information do not always align. LedgerLine uses a deterministic reconciliation layer as the source of truth and then adds machine-learning-based prioritization to help analysts decide which exceptions deserve attention first.

The current implementation works with a synthetic dataset of 500 financial transactions. Input records are validated and normalized before reconciliation. The deterministic engine checks transaction identity, references, amounts, dates, payment status, ledger consistency, and duplicate conditions. It can produce outcomes such as MATCHED, DATE_MISMATCH, DUPLICATE_PAYMENT, AMOUNT_MISMATCH, LEDGER_EXCEPTION, PAYMENT_FAILED, and PAYMENT_REFUNDED. These outcomes are then classified into operational exceptions.

A Logistic Regression model provides an interpretable risk layer. It produces a risk probability, unified risk score, risk level, and top risk factors. The model is intentionally separated from the financial truth: it prioritizes operational attention but does not overwrite the final reconciliation status. Gemini can be configured as an investigation assistant. It receives structured transaction evidence already available through the API and explains the evidence rather than independently deciding what happened.

The application is implemented using Python, FastAPI, Pandas, scikit-learn and Google Gemini on the backend, with React and Vite on the frontend. The API exposes health, summary, transaction, exception, and investigation endpoints. Automated testing uses Pytest and Vitest, while the project also includes a transaction-level benchmark verification script. The result is a reproducible academic project that demonstrates how deterministic business rules, interpretable ML, and evidence-grounded generative AI can be combined while keeping human review in the decision loop.

Keywords: Finance Operations, Reconciliation, Exception Management, Logistic Regression, Risk Prioritization, FastAPI, React, Gemini, Synthetic Financial Data, Evidence-Grounded AI.

2. INTRODUCTION OF PROJECT

LedgerLine was developed as a practical finance-operations system rather than as a generic chatbot or an autonomous financial decision maker. The central idea is to separate three responsibilities that are often mixed together in AI prototypes. First, deterministic logic establishes what the supplied transaction evidence says. Second, machine learning helps prioritize which cases require attention. Third, generative AI can help an analyst understand or summarize the evidence that already exists. This separation makes the workflow easier to audit and easier to test.

The project begins with synthetic financial source records. The data pipeline validates and normalizes the input before any decision is produced. The reconciliation engine then compares transaction identity, references, amounts, dates, payment status and ledger consistency. Duplicate conditions are also considered. The resulting status becomes the operational baseline for the rest of the system.

Once exceptions are identified, the ML layer estimates which records should receive higher operational attention. The current model is Logistic Regression because it is relatively easy to interpret and suitable for demonstrating a probability-based risk score. The project explicitly avoids treating the model as the source of financial truth. This is important because the present target label is derived from reconciliation rules, meaning the model evaluation is a pipeline sanity check rather than proof of production predictive performance.

The frontend converts the backend outputs into an operations control center. The dashboard brings together KPI information, exception and risk distributions, settlement-delay analysis, a priority action queue, transaction exploration, transaction details, ML risk intelligence, and investigation provenance. The aim is to reduce the need to inspect multiple raw files or API responses while preserving the evidence behind each result.

2.1 BACKGROUND

Financial reconciliation is the process of checking whether records representing the same economic event agree across different systems. In a simplified setting, a transaction may appear in a bank record, a payment gateway record and an internal ledger. Each source can use different identifiers, dates or statuses. Even when the underlying payment is valid, a mismatch in one field can create an exception that needs human review.

Manual investigation becomes difficult when exception volume grows. Analysts have to compare records, identify the type of mismatch, decide which cases are more urgent, and document why a case was reviewed. A useful operational platform therefore needs more than a matching rule. It needs a clear workflow for validation, classification, prioritization and investigation.

LedgerLine uses synthetic data so that the entire pipeline can be reproduced without exposing real financial information. The repository contains 500 synthetic transactions, raw records under data/raw/ and processed application outputs under data/processed/. Small trained model artifacts are stored under models/ so that inference can run without retraining.

2.2 PROBLEM STATEMENT

The problem addressed by LedgerLine can be stated as follows: financial transaction records from multiple sources need to be reconciled consistently, exceptions need to be classified, and analysts need a transparent way to prioritize and investigate those exceptions. A system that simply predicts a label is insufficient because financial operations require traceability and evidence.

- Different source records may disagree on identifiers, dates, amounts or payment status.
- Duplicate payments and ledger exceptions can require separate operational treatment.
- Large exception queues make it difficult to decide what should be reviewed first.
- Machine-learning predictions should not silently replace deterministic financial rules.
- Generative AI should be constrained to supplied evidence instead of becoming an unsupported source of financial truth.
- Analysts need a single dashboard that exposes results, risk factors and investigation provenance.

3. OBJECTIVES AND KEY LEARNINGS

3.1 OBJECTIVES

1. Build a reproducible finance-operations pipeline using synthetic financial records.
2. Validate and normalize source data before applying reconciliation logic.
3. Implement deterministic reconciliation as the primary decision layer.
4. Classify operational exceptions into understandable categories.
5. Add an interpretable ML layer for attention prioritization rather than financial decision replacement.
6. Expose the pipeline through a typed REST API using FastAPI.
7. Develop a React/Vite operations dashboard for reviewing KPIs, exceptions and transactions.
8. Integrate Gemini as an evidence-grounded investigation assistant with server-side credentials.
9. Provide deterministic fallback behavior when the external AI provider is unavailable.
10. Validate the implementation through backend tests, frontend tests/build checks and benchmark verification.

3.2 KEY LEARNINGS

- Business rules and ML can coexist when their responsibilities are explicitly separated.
- Interpretability matters in finance-oriented workflows; risk factors are more useful when analysts can understand why attention is high.
- Synthetic data is useful for reproducibility, but synthetic evaluation should not be presented as production model performance.
- AI integrations need provenance, validation and fallback behavior rather than a single dependency on an external provider.
- An API-first backend creates a clean boundary between data processing and the user interface.
- Testing needs to cover not only individual functions but also the final decision path and application build.

4. FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Functional Requirements

ID	Requirement	Description
FR-01	Data ingestion	Load synthetic financial source records for processing.
FR-02	Validation	Check and normalize incoming fields before reconciliation.
FR-03	Reconciliation	Compare identity, references, amounts, dates, status and ledger consistency.
FR-04	Exception classification	Assign operational outcomes such as DATE_MISMATCH or AMOUNT_MISMATCH.
FR-05	Risk prioritization	Generate interpretable ML risk probability, score, level and factors.
FR-06	REST API	Serve health, summary, transaction, exception and investigation endpoints.
FR-07	Dashboard	Present KPIs, distributions, queues, transaction exploration and detail views.
FR-08	Investigation	Provide evidence-grounded investigation assistance using Gemini when configured.
FR-09	Fallback	Use a deterministic evidence-based response when Gemini is unavailable.
FR-10	Testing	Run backend/frontend tests and benchmark verification.

Non-Functional Requirements

Category	Requirement
Reproducibility	The pipeline should work from a known synthetic dataset and stored model artifacts.
Auditability	Final reconciliation status must remain traceable to deterministic rules.
Interpretability	Risk prioritization should expose understandable factors.
Security	Gemini credentials must remain server-side and outside the frontend.
Maintainability	Backend, frontend, data, ML and AI responsibilities should remain separated.
Usability	The dashboard should make exception review easier than inspecting raw records.
Reliability	External AI availability should not be required for local deterministic operation.
Performance	The API should provide structured results suitable for interactive dashboard use.

5. HIGH LEVEL AND LOW LEVEL DESIGN

5.1 HIGH-LEVEL DESIGN

The high-level architecture follows a pipeline with a controlled flow of responsibility. Raw synthetic data first enters validation and normalization. The normalized records are passed to deterministic reconciliation, which produces the source-of-truth status. Exceptions are then classified and sent to the ML risk layer for attention prioritization. FastAPI exposes the resulting information to the React/Vite dashboard. Gemini is connected as an optional investigation assistant, receiving structured evidence already exposed by the application.

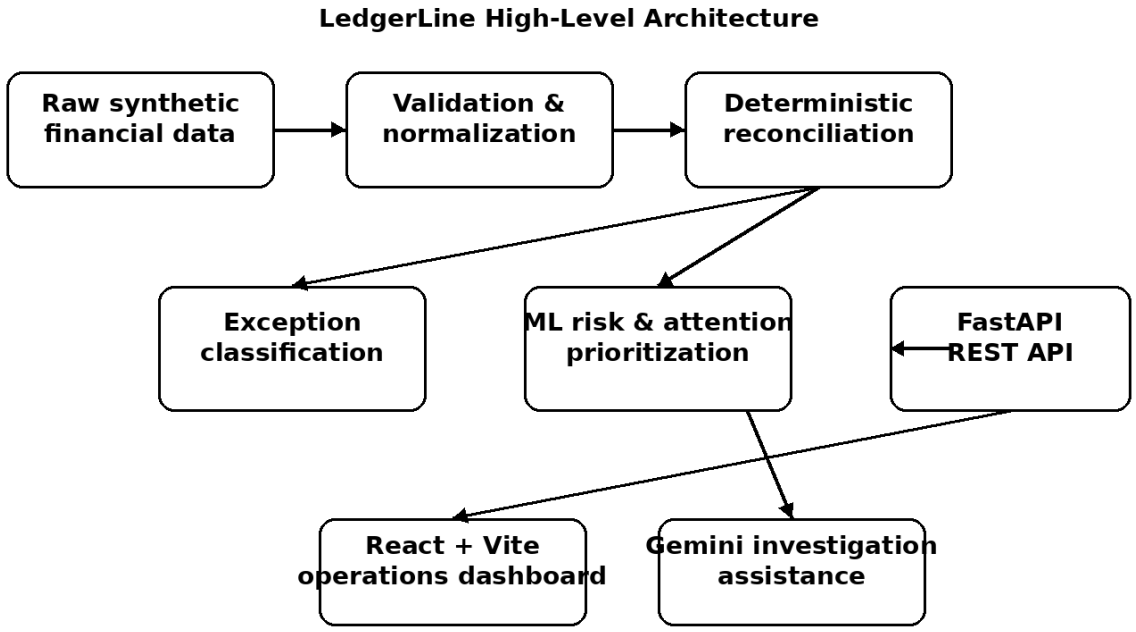

Figure 5.1: High-level LedgerLine architecture based on the implemented repository workflow.

A key design decision is the direction of authority. The deterministic reconciliation engine decides what happened according to the implemented business rules. ML decides how strongly an analyst should be encouraged to look at the case. Gemini helps explain the supplied evidence. The human analyst remains responsible for the operational decision.

5.2 DESIGN PRINCIPLES

- Deterministic-first decisioning.
- Separation of truth, prioritization and explanation.
- Evidence-grounded AI rather than open-ended financial advice.
- Server-side handling of provider credentials.

- Reproducible synthetic data and stored inference artifacts.
- Clear frontend/backend separation through REST + JSON.

5.3 LOW-LEVEL DESIGN

At the lower level, the project is organized around several cooperating responsibilities. The data layer reads raw CSV records and prepares processed records. Reconciliation logic applies the transaction comparison rules and generates the final status. Exception classification translates reconciliation outcomes into operational categories. The ML layer consumes permitted features and returns risk outputs. The API layer packages these outputs into JSON responses, while the frontend renders them into dashboard components.

Layer	Main responsibility	Representative artifact
Data	Load, validate and normalize records	data/raw/, data/processed/
Reconciliation	Determine transaction outcome	reconciliation logic under src/
ML	Estimate attention/risk priority	models/risk_model.joblib
AI	Explain supplied transaction evidence	Gemini investigation service
API	Expose structured services	src/api/main.py
Frontend	Visualize operations information	frontend/ React/Vite app
Testing	Regression and benchmark validation	tests/, src/evaluation/

5.4 DATA FLOW

1. Read synthetic source records.
2. Validate required fields and normalize representations.
3. Apply deterministic reconciliation checks.
4. Store or expose final status and exception information.
5. Calculate ML-based risk probability and attention level.
6. Expose the results through FastAPI.
7. Render KPIs, queues and transaction details in React.
8. Optionally send structured evidence to Gemini for investigation assistance.
9. Display provider provenance and retain deterministic fallback behavior.

6. PROGRAM'S STRUCTURE ANALYSING AND GUI CONSTRUCTING

The repository is organized so that the data pipeline, model artifacts, application code, frontend and tests remain visibly separated. This structure is suitable for an academic project because a reader can identify where data processing happens, where model artifacts are stored, where the API is defined, and where the dashboard is implemented.

Directory	Purpose
data/	Synthetic raw and processed financial records.
models/	Trained ML artifacts and model metadata.
src/	Data processing, reconciliation, ML, AI, API and evaluation code.
frontend/	React/Vite operations dashboard.
tests/	Automated backend and API regression tests.

6.1 MAIN PROGRAM MODULES

- Data processing modules: responsible for reading and preparing the synthetic source records.
- Reconciliation modules: implement deterministic comparison and final-status logic.
- ML modules: train/load the Logistic Regression model and produce risk outputs.
- AI modules: prepare structured evidence and validate investigation responses.
- API modules: expose the service endpoints used by the dashboard.
- Evaluation modules: verify benchmark behavior and report mismatches separately.
- Frontend components: render KPI cards, distributions, queues, transaction views and investigation information.

6.2 GUI CONSTRUCTION

The GUI is designed as an operations control center. Instead of presenting a single prediction screen, it organizes information around the work an analyst needs to perform: understand the overall state, identify exceptions, prioritize cases, inspect transaction evidence and request an investigation explanation.

GUI SNAPSHOT

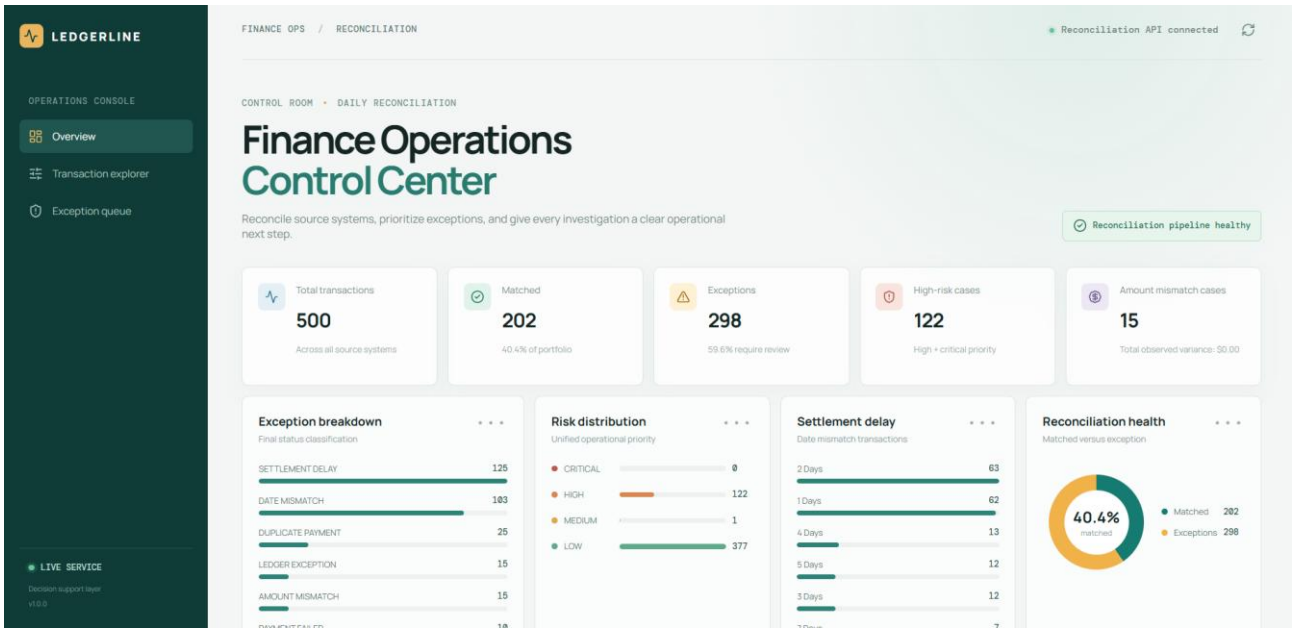

Figure 6.1: Schematic representation of the implemented operations control-center layout. It is a report illustration of the documented dashboard modules, not a browser screenshot.

The dashboard described in the project documentation includes a KPI overview, exception and risk distribution, settlement-delay analysis, a priority action queue, a transaction explorer, a transaction detail drawer, ML risk intelligence and Gemini investigation provenance. These modules are intentionally arranged around the analyst's workflow rather than around the internal code structure.

6.3 USER WORKFLOW

1. Open the operations dashboard and review the KPI summary.
2. Inspect the exception and risk distributions to understand the current workload.
3. Open the priority action queue and select a high-attention case.
4. Use the transaction explorer/detail view to inspect the supplied evidence.
5. Review the ML risk probability, risk level and top risk factors.
6. Request an investigation explanation when required.
7. Use the returned explanation together with the deterministic status to support the human review decision.

The interface therefore acts as a presentation and investigation layer over the underlying deterministic pipeline. It does not change the source-of-truth status simply because an AI explanation is generated.

INVESTIGATION VIEW

6.4 INVESTIGATION ASSISTANCE

Gemini is integrated as an investigation assistant. The server sends only structured transaction evidence that is already available through the application API. This keeps the AI request bounded by the evidence used by the application. The response is schema-validated, and the dashboard displays provider provenance so that a user can distinguish an AI-generated investigation from a deterministic fallback.

Condition	System behavior
Gemini configured and available	Structured transaction evidence is sent to the configured Gemini provider and the returned response is validated before display.
Gemini not configured	The application can operate in deterministic mode for offline/local demonstrations.
Gemini unavailable or fails	A deterministic evidence-based fallback can support the local workflow.
API credentials	Stored server-side through environment configuration; not exposed to React.
Decision authority	Gemini does not replace final_status or the existing recommended action.

6.5 WHY THIS GUI APPROACH IS SUITABLE

- It keeps high-level workload information visible before individual investigation.
- It provides a queue-oriented view for operational prioritization.
- It allows analysts to move from a transaction list to transaction-level evidence.
- It exposes risk factors rather than hiding ML behavior behind a single score.
- It makes AI provenance visible and preserves deterministic fallback behavior.

From an academic perspective, the GUI demonstrates the complete path from data processing to an operational user experience. The interface is therefore not only a visual layer; it is the final stage of the system's information flow.

7. CODE-IMPLEMENTATION AND DATABASE CONNECTIONS

7.1 BACKEND IMPLEMENTATION

The backend is implemented in Python using FastAPI. FastAPI provides typed REST endpoints and validation while keeping the application boundary clear for the React frontend. The project exposes endpoints for service health, summary information, paginated and filtered transactions, transaction details, prioritized exceptions and evidence-grounded investigation.

Method	Endpoint	Purpose
GET	/health	Service health and available record count.
GET	/summary	KPI and distribution summary.
GET	/transactions	Paginated and filtered transaction records.
GET	/transactions/{transaction_id}	Full transaction detail.
GET	/exceptions	Prioritized exception records.
POST	/transactions/{transaction_id}/investigate	Evidence-grounded investigation.

7.2 DATA STORAGE / DATABASE CONNECTIONS

The current implementation does not use a relational database as its primary application store. The project deliberately uses CSV-based synthetic data with Pandas so that the data pipeline remains transparent and reproducible for the academic demonstration. Raw records are kept under data/raw/ and processed outputs under data/processed/. This design should therefore be described as a file-based data layer rather than as a conventional database connection.

This choice is consistent with the project's stated scope. The purpose is to demonstrate reconciliation, ML prioritization, API delivery and AI-assisted investigation without introducing unnecessary database infrastructure. A production version could replace or supplement the CSV layer with a transactional database and persistent audit storage.

ML IMPLEMENTATION

7.3 MACHINE LEARNING IMPLEMENTATION

The selected model is Logistic Regression. The model's role is deliberately limited to operational attention prioritization. It produces a risk probability, a unified risk score, a risk level and top risk factors that can be displayed to an analyst.

The project documentation states that identity fields, deterministic outcome fields and evaluation-only fields are excluded from training features. The current high_attention target is derived from reconciliation rules and trained using 500 synthetic records. This means the present evaluation should be interpreted as a pipeline sanity check. It should not be described as evidence that the model will predict real-world financial risk accurately.

ML Element	Implementation choice
Model	Logistic Regression
Training data	500 synthetic records
Target	high_attention derived from reconciliation rules
Output	Risk probability, unified score, risk level and top factors
Excluded information	Identity, deterministic outcome and evaluation-only fields
Stored artifacts	risk_model.joblib, metadata, metrics, feature importance and baseline comparison
Production limitation	Requires historical analyst outcomes for a production-grade target

7.4 MODEL GOVERNANCE PRINCIPLE

The most important implementation principle is that the ML model cannot overwrite the deterministic final decision. This avoids turning a probabilistic prioritization model into an unsupported financial decision engine. In a future production setting, historical analyst outcomes such as review, escalation, confirmed resolution, SLA breach or financial loss could be used to create a more meaningful target.

AI AND DEPLOYMENT

7.5 GEMINI INTEGRATION

Gemini is used for evidence-grounded investigation assistance. The provider is configured through environment variables. The project documentation specifies that the API key remains server-side and should never be placed in the React frontend. The application can also run with a deterministic provider for offline operation.

- AI_INVESTIGATION_PROVIDER controls the investigation provider.
- GEMINI_API_KEY is configured on the server side.
- GEMINI_MODEL specifies the selected Gemini model.
- GEMINI_TIMEOUT_SECONDS controls the request timeout.
- The response is schema-validated before being surfaced to the dashboard.

7.6 DEPLOYMENT

The repository contains deployment configuration for a Render FastAPI service and a Vercel Vite dashboard. The production API is started with Uvicorn, while the frontend receives the public API base URL through VITE_API_BASE_URL. The deployment design keeps the Gemini secret on the backend and exposes only the API base URL to the frontend.

Component	Deployment role
Render	Hosts the FastAPI backend service.
Vercel	Builds and serves the React/Vite dashboard.
Environment variables	Provide CORS, provider selection, Gemini credentials/model and frontend API base URL.
Health check	/health endpoint is used for service health.
Frontend security rule	GEMINI_API_KEY must not be set in Vercel.

7.7 LOCAL EXECUTION

The documented local workflow creates a Python virtual environment, installs requirements, configures .env, starts FastAPI with Uvicorn, installs frontend dependencies and runs the Vite development server. API documentation is available through FastAPI's /docs endpoint.

8. SYSTEM TESTING

Testing in LedgerLine is designed to check the system at multiple levels. Backend and API behavior is covered with Pytest. The frontend uses Vitest for test coverage and a production build command checks whether the Vite application can be bundled successfully. The project also contains a benchmark verification script that checks transaction-level final decisions.

8.1 TESTING COMMANDS

Test area	Command / check	Purpose
Backend	python -m pytest	Run backend and API regression tests.
Frontend tests	npm test	Run frontend test suite.
Frontend build	npm run build	Verify production bundle generation.
Benchmark	python src/evaluation/verify_benchmark.py	Verify transaction-level benchmark behavior.
API smoke check	GET /health	Confirm service is responding.
Summary endpoint	GET /summary	Confirm KPI/summary response.
Investigation endpoint	POST /transactions/{id}/investigate	Validate investigation workflow.

8.2 TESTING RESULT INTERPRETATION

According to the project documentation, the current benchmark reports zero final-decision mismatches. Duplicate payment assignment differences are reported separately. This distinction matters because it shows that the evaluation is checking the final decision path while also surfacing a specific area where assignment-level differences can be inspected.

8.3 TESTING LIMITATIONS

- The dataset is synthetic, so test success does not prove behavior on real financial records.
- The ML target is derived from business rules, limiting the meaning of predictive evaluation.
- External Gemini behavior depends on provider availability and configuration.
- Production-scale performance, security and database durability are outside the current demonstration scope.

9. ADVANTAGES AND DISADVANTAGES

9.1 ADVANTAGES

- Deterministic reconciliation provides a clear and auditable source of truth.
- Exception categories make operational problems easier to organize.
- ML risk prioritization helps analysts focus on cases that require more attention.
- Risk factors improve interpretability compared with exposing only a probability.
- Gemini can accelerate evidence-based investigation without becoming the financial decision maker.
- FastAPI and React provide a clean separation between backend processing and frontend presentation.
- The synthetic dataset makes the project reproducible without using real customer financial data.
- Stored model artifacts allow inference without retraining.
- Deterministic fallback behavior keeps the core workflow usable when Gemini is unavailable.
- Automated tests and benchmark verification provide repeatable quality checks.

9.2 DISADVANTAGES / CURRENT LIMITATIONS

- The dataset is synthetic and therefore does not represent the full variability of real financial operations.
- The current ML target is generated from rules rather than historical analyst outcomes.
- The system is not a real-time financial processing platform.
- The current data layer is file-based rather than a production transactional database.
- Gemini is an optional external dependency and its output must still be reviewed by a human.
- The current project does not provide a complete analyst resolution workflow or audit timeline.
- Production deployment would require stronger operational controls, access management, monitoring and persistent audit infrastructure.

Overall, the limitations are explicit rather than hidden. This is important because the project is intended as a technical demonstration and academic implementation, not as a production financial system.

10. CONCLUSION AND FUTURE IMPROVEMENTS

10.1 CONCLUSION

LedgerLine demonstrates a practical way to combine deterministic financial processing, interpretable machine learning and generative AI within a single finance-operations workflow. The system begins with synthetic transaction records, validates and normalizes them, applies deterministic reconciliation, classifies exceptions, prioritizes attention using Logistic Regression, exposes the results through FastAPI and presents them through a React/Vite dashboard.

The strongest architectural decision is the separation of responsibilities. Deterministic logic remains the source of truth; ML supports prioritization; Gemini supports explanation; and the human analyst remains responsible for the operational decision. This makes the system easier to reason about than an approach in which a single AI model is asked to make all financial decisions.

The project also demonstrates the importance of honest evaluation. Because the current ML target is derived from business rules and the dataset is synthetic, the reported evaluation is appropriately treated as a pipeline sanity check. This distinction prevents the project from overstating what its current model can prove.

10.2 FUTURE IMPROVEMENTS

- Add an analyst resolution workflow with review, escalation and closure states.
- Maintain a detailed audit timeline for each transaction investigation.
- Support upload-and-rerun reconciliation jobs for new source files.
- Train the attention model on historical analyst outcomes rather than rule-derived labels.
- Introduce a production database and durable audit storage.
- Add role-based access and stronger deployment security controls.
- Expand monitoring, logging and operational metrics for deployed services.
- Improve evaluation with realistic historical datasets and time-based validation.
- Add richer evidence views and controlled analyst feedback loops.
- Package environment-specific deployment and operational configuration more fully.

The project therefore provides a solid base for extending an academic prototype into a more complete finance-operations application while retaining the same deterministic-first philosophy.

11. REFERENCES

The following references are the primary project and implementation materials used for this report. The report intentionally keeps the technical claims aligned with the supplied project documentation rather than introducing unrelated product descriptions or external benchmarks.

1. LedgerLine – Finance Operations Project Repository. GitHub repository containing the source code, project structure, synthetic dataset, ML artifacts, backend API, frontend dashboard and deployment configuration. https://github.com/kritika0519/LedgerLine-Finance-ops
2. Python Documentation. Python Software Foundation. Used as the primary programming language for data processing, backend development, machine learning and AI integration. https://docs.python.org/3/
3. Pandas Documentation. Used for tabular data processing, CSV handling, data cleaning and transformation in the LedgerLine data pipeline. https://pandas.pydata.org/docs/
4. FastAPI Documentation. Used for developing the REST API, request handling, validation and API documentation for the LedgerLine backend. https://fastapi.tiangolo.com/
5. scikit-learn Documentation. Used for implementing the Logistic Regression model for ML-based risk and attention prioritization. https://scikit-learn.org/
6. React Documentation. Used for developing the frontend operations dashboard and component-based user interface. https://react.dev/
7. Vite Documentation. Used as the frontend development server and build tool for the React application. https://vite.dev/
8. Google Gemini API Documentation. Used for the optional evidence-grounded investigation assistant integrated into LedgerLine. https://ai.google.dev/gemini-api/docs
9. Pytest Documentation. Used for automated backend and API testing of the LedgerLine application. https://docs.pytest.org/
10. Vitest Documentation. Used for frontend testing and regression checks for the React/Vite dashboard. https://vitest.dev/

APPENDIX – KEY PROJECT ARTIFACTS

A. TECHNOLOGY STACK

Layer	Technology	Role
Frontend	React + Vite	Component-based operations dashboard.
Styling	CSS	Lightweight responsive enterprise UI.
Backend	FastAPI	Typed REST API and validation.
Language	Python	Data processing, ML, API and AI integration.
Data	Pandas + CSV	Transparent, reproducible data pipeline.
ML	scikit-learn	Interpretable risk/attention modeling.
AI	Google Gemini	Evidence-grounded investigation assistance.
Testing	Pytest + Vitest	Backend/frontend regression coverage.
API	REST + JSON	Frontend/backend separation.

B. IMPORTANT REPOSITORY ARTIFACTS

- data/raw/ – synthetic raw financial records.
- data/processed/ – processed application outputs.
- models/risk_model.joblib – stored model artifact for inference.
- models/risk_model_metadata.json – model metadata.
- models/risk_model_metrics.json – model evaluation information.
- models/feature_importance.csv – feature importance information.
- models/risk_baseline_vs_ml.json – baseline versus ML comparison.
- src/ – data, reconciliation, ML, AI, API and evaluation implementation.
- frontend/ – React/Vite dashboard.
- tests/ – automated test suite.

C. DOCUMENTED API SURFACE

GET /health, GET /summary, GET /transactions, GET /transactions/{transaction_id}, GET /exceptions, and POST /transactions/{transaction_id}/investigate form the documented API surface for the current implementation.

D. PROJECT SCOPE NOTE

LedgerLine is an academic demonstration built around synthetic data. It is not presented as a real-time financial processing platform, and its ML layer is not claimed to have production predictive performance. These boundaries are part of the project's technical correctness.